

PANEL BACKTESTING

Documentación Técnica Completa del Sistema

Arquitectura · Módulos · Flujo de datos · Estrategias · Métricas · Reportes

Versión del documento: June 2026 · Para uso personal

Índice de contenidos

1. Visión general del sistema
2. Flujo de datos de extremo a extremo
3. CONFIGURACION — config.py
4. DATOS — Carga, resampleo y validación
 - 4.1 cargador.py
 - 4.2 tiempo.py
 - 4.3 resampleo.py
 - 4.4 validador.py
 - 4.5 perturbaciones.py (Monte Carlo)
5. MOTOR — El núcleo Rust
6. NUCLEO — La capa de orquestación Python
 - 6.1 base_estrategia.py
 - 6.2 contexto.py
 - 6.3 proyeccion.py
 - 6.4 integridad.py
 - 6.5 registro.py
7. ESTRATEGIAS — Las cinco estrategias disponibles
8. SALIDAS — Los cuatro tipos de cierre
9. OPTIMIZACION — El cerebro del sistema
 - 9.1 runner.py — Orquestador principal
 - 9.2 metricas.py — Puente Rust → Python
 - 9.3 puntuacion.py — Función objetivo
 - 9.4 samplers.py — Algoritmos de búsqueda
10. COMUN — Utilidades compartidas
11. REPORTEES — Todo lo que se genera
 - 11.1 analitica.py — Fuente única de métricas
 - 11.2 reporte_pdf.py — El informe en PDF
 - 11.3 excel.py — Informe Excel
 - 11.4 html.py — Gráficos interactivos
 - 11.5 informe.py — Paisaje de parámetros
 - 11.6 persistencia.py — Archivos permanentes
 - 11.7 rich.py — Terminal en tiempo real
 - 11.8 metricas_presentacion.py — Etiquetas y juicios
12. Guía completa de métricas
13. Limitación In-Sample y próximos pasos

1. Visión general del sistema

PANEL BACKTESTING es un sistema de backtesting cuantitativo personal diseñado para probar, optimizar y evaluar estrategias de inversión algorítmica sobre datos históricos de precio. El objetivo es que los resultados que produce sean lo más fieles posible a lo que ocurriría en mercados reales, evitando los errores estadísticos más comunes que distorsionan las conclusiones.

El sistema puede trabajar simultáneamente con cinco activos de naturaleza muy distinta —Oro, Plata, Bitcoin, S&P 500 y Nasdaq 100— y con cualquier combinación de timeframe y tipo de salida. Para cada combinación, lanza una optimización con Optuna que busca los parámetros que maximizan la función objetivo elegida (por defecto, el PSR) y genera automáticamente un informe PDF profesional con todas las métricas.

Características técnicas principales:

- Motor de simulación escrito en Rust (compilado como extensión Python vía PyO3), lo que lo hace extremadamente rápido y permite ejecutar cientos de trials de Optuna en paralelo.
- Cero copia de datos entre Python y Rust: los arrays numpy se pasan directamente a memoria Rust sin duplicar, lo que ahorra tiempo y memoria en optimizaciones largas.
- Pipeline completo con más de 40 métricas de rendimiento y riesgo, desde el Sharpe anualizado hasta el SQN de Van Tharp o el Z-Score de Wald–Wolfowitz.
- Sistema de caché de indicadores (LRU, 512 MB) que evita recalcular indicadores caros entre trials de Optuna cuando los parámetros de los indicadores no cambian.
- Cuatro tipos de salida: Stop/TP fijo, número de velas, trailing stop y salida personalizada por estrategia.
- Perturbaciones Monte Carlo: cada trial puede ver una variante aleatoria distinta de los datos históricos, reduciendo el sobreajuste a una sola trayectoria de precios.
- Detección automática de régimen de mercado y modo híbrido activo/comprar-y-mantener.
- Verificación de determinismo en replay: el sistema re-ejecuta los mejores trials y comprueba que los resultados coinciden exactamente con los de la optimización.

Estructura de carpetas:

Carpeta	Función principal	Archivos clave
CONFIGURACION/	Parámetros globales del sistema	config.py
DATOS/	Carga, normalización, resampleo y validación de datos históricos	cargador · tiempo · resampleo · validador · perturbaciones
MOTOR/	Puente Python↔Rust para el motor de simulación compilado	wrapper.py
NUCLEO/	Clase base de estrategias, contexto de datos y verificaciones	base_estrategia · contexto · proyeccion · integridad · registro
ESTRATEGIAS/	Cada archivo es una estrategia concreta que hereda de BaseEstrategia	ema_tendencia · rsi_reversion · vat_absorcion · vwap_distance · vwapcvd
SALIDAS/	Configuración de los modos de cierre de operaciones	fijo · velas · trailing · personalizada
OPTIMIZACION/	Orquestador principal, función objetivo y samplers de Optuna	runner · metricas · puntuacion · samplers

Carpeta	Función principal	Archivos clave
COMUN/	Fórmulas estadísticas y utilidades numpy compartidas por todo el sistema	estadistica · numpy_utiles
REPORTES/	Todo lo que se genera al finalizar una optimización	analitica · reporte_pdf · excel · html · informe · persistencia · rich · ...
HISTORICO/	Datos históricos OHLCV en formato parquet / feather / csv	BTC_2021-2024_1h.parquet, etc.

2. Flujo de datos de extremo a extremo

Este diagrama describe qué ocurre desde que el usuario ejecuta el sistema hasta que se genera el PDF de resultados. Cada flecha representa una transferencia de datos o una llamada entre módulos.

PASO 1	Carga de configuración	El usuario ejecuta <code>runner.py</code> . Se lee <code>config.py</code> y se construye el objeto de configuración con todos los parámetros: activos, timeframes, fechas, tipo de salida, número de trials, función de score, etc. Si algo está mal configurado, el sistema para con un mensaje claro antes de hacer nada.
PASO 2	Registro automático de estrategias	<code>registro.py</code> escanea la carpeta <code>ESTRATEGIAS/</code> e importa todos los archivos <code>.py</code> . Cualquier clase que herede de <code>BaseEstrategia</code> queda registrada automáticamente por su ID. No hay que declarar nada a mano.
PASO 3	Carga y validación de datos	<code>cargador.py</code> localiza el archivo de menor timeframe disponible para cada activo en <code>HISTORICO/</code> (parquet, feather o csv), lo carga con Polars, normaliza el timestamp a UTC microsegundos y filtra el rango de fechas. <code>validador.py</code> comprueba columnas mínimas, valores nulos, orden cronológico, duplicados, huecos temporales y coherencia OHLC. Si hay problemas, para con detalle del error.
PASO 4	Resamdeo y construcción del contexto	<code>resamdeo.py</code> construye el timeframe pedido (ej. 4h) a partir del base (1h) con reglas explícitas por columna: <code>open=primero</code> , <code>close=último</code> , <code>high=máximo</code> , <code>low=mínimo</code> , <code>volume=suma</code> . <code>contexto.py</code> materializa los arrays numpy contiguos una sola vez para toda la optimización, y <code>proyeccion.py</code> construye el índice de mapeo TF-estrategia → TF-base para la regla N+1.
PASO 5	Optimización con Optuna	<code>runner._optimizar_combinacion()</code> crea un estudio Optuna con el sampler elegido (QMC, TPE o HYBRID). Por cada trial: la estrategia propone señales (-1, 0, +1), se proyectan al timeframe base, el motor Rust las simula y devuelve las Métricas. La función objetivo (<code>puntuacion.py</code>) calcula el score. Optuna decide el siguiente punto del espacio de búsqueda. El cache de indicadores evita recálculos redundantes entre trials que comparten parámetros de indicadores.
PASO 6	Replay de los mejores trials	Al terminar la optimización, <code>runner._replay_trial()</code> re-ejecuta los top-N trials con <code>simular_full()</code> para obtener el historial completo de trades y la curva de equity. <code>integridad.py</code> verifica que los resultados del replay coinciden exactamente (tolerancia 1e-7) con los de la optimización: número de trades y saldo final deben ser idénticos. Si no coinciden, hay un bug de determinismo.
PASO 7	Generación de métricas avanzadas	<code>analitica.py</code> recibe los trades y la equity curve del replay y calcula más de 40 métricas: Sortino, PSR, SQN, Z-Score, R-Expectancy, AHPG/GHPR, Stagnation, VaR/CVaR, returns mensuales, etc. <code>metricas_presentacion.py</code> añade etiquetas, formatos y valoraciones (BUENO/REGULAR/MALO).
PASO 8	Persistencia y reportes	<code>persistencia.py</code> guarda en disco: <code>resumen.json</code> , <code>auditoria.json</code> , <code>trials.csv</code> , <code>trades.csv</code> , <code>equity.csv</code> . <code>reporte_pdf.py</code> genera el PDF profesional de 2 páginas con KPIs, estadísticas y tabla de retornos mensuales. <code>excel.py</code> genera el Excel con hoja por trial. <code>html.py</code> genera hasta 5 gráficos interactivos TradingView. <code>informe.py</code> genera el informe HTML del paisaje de parámetros. <code>rich.py</code> muestra el resumen en terminal con colores.

3. CONFIGURACION — config.py

config.py es el único lugar donde el usuario toca parámetros. Cualquier otra parte del sistema lee de aquí. Si se necesita cambiar el activo, las fechas o el número de trials, se hace en este archivo y nada más.

Parámetro	Valor ejemplo	Qué hace
ACTIVOS	["BTC"]	Lista de activos a probar. Cada nombre debe coincidir con un archivo en HISTORICO/.
FORMATO_DATOS	"parquet"	Formato del archivo histórico: parquet, feather o csv.
MERCADO_24_7	{"BTC": True}	Si True, el validador exige continuidad de velas sin huecos. Si False (ej. Oro), permite sesiones.
TIMEFRAMES	["1h"]	Timeframes de la estrategia. Los datos se resamplean desde el timeframe base (1h).
FECHA_INICIO FECHA_FIN	/ "2021-01-01" "	Rango de fechas sobre el que se trabaja. Solo se cargan velas dentro de este intervalo.
ESTRATEGIA_ID	1	ID de la estrategia a ejecutar. Puede ser 1, [1,2], o "all" para todas las disponibles.
SALDO_INICIAL	10 000	Capital de partida de la simulación en unidades monetarias (p.ej. USDT).
SALDO_USADO_POR_TRADE	500	Colateral que se asigna a cada operación. Determina el tamaño de posición junto con el apalancamiento.
APALANCAMIENTO	8	Multiplicador de exposición. Posición real = colateral × apalancamiento.
COMISION_PCT	0.0005	Comisión porcentual por lado (0.05% = 5 puntos básicos). Se cobra en entrada y salida.
COMISION_LADOS	2	Número de lados en los que se cobra comisión. 2 = entrada + salida (estándar).
EXIT_TYPE	"BARS"	Modo de cierre activo: FIXED, BARS, TRAILING o CUSTOM.
N_TRIALS	300	Número de evaluaciones que hace Optuna. Más trials = mejor exploración, pero más tiempo.
OPTUNA_SAMPLER	"QMC"	Algoritmo de búsqueda: QMC (cuasi-aleatorio), TPE (bayesiano) o HYBRID (mitad+mitad).
FUNCION_SCORE	"PSR"	Métrica que Optuna maximiza: PSR, SHARPE, CALMAR o ROI.
MIN_TRADES_SCORE	30	Si un trial tiene menos trades que este mínimo, su score es 0. Evita estrategias con poquísimas operaciones.
USAR_SEED	False	Si True, la búsqueda de Optuna es reproducible con la semilla fijada. Útil para comparar runs.
PERTURBACIONES_ACTIVAS	False	Si True, cada trial ve datos con pequeñas perturbaciones aleatorias Monte Carlo. Reduce sobreajuste.
N_JOBS	-2	-2 = todos los núcleos menos uno. Controla el paralelismo de Optuna.

4. DATOS — Carga, resampleo y validación

La carpeta DATOS/ agrupa todo lo relacionado con los datos históricos: cómo se localizan, cómo se normalizan, cómo se resamplean a un timeframe mayor y cómo se valida su integridad antes de usarlos. También contiene el módulo de perturbaciones Monte Carlo.

4.1 cargador.py

`cargador.cargar(activo, cfg)` es el punto de entrada para los datos. Hace tres cosas en orden:

Primero, **localiza el archivo** más de menor timeframe disponible para el activo en la carpeta HISTORICO/. El nombre del archivo debe seguir el patrón `ACTIVO-fechas-TIMEFRAME.parquet`, por ejemplo `BTC_2021-2024_1h.parquet`. Si hay más de un archivo que encaja, el sistema para con un error para evitar ambigüedad.

Segundo, **normaliza el timestamp** a `Datetime(us, UTC)` con microsegundos epoch independientemente del formato original del archivo (puede venir en nanosegundos, sin zona horaria, como string, etc.). Esto garantiza que todo el sistema trabaja siempre con la misma representación temporal.

Tercero, **filtra el rango de fechas** según `FECHA_INICIO` y `FECHA_FIN` de la configuración. Si el filtro no deja ninguna fila, para con un error descriptivo.

4.2 tiempo.py

Es la fuente única de verdad sobre timeframes. Define la jerarquía ordenada de menor a mayor (1m → 5m → 15m → 30m → 1h → 4h → 1d), los segundos que dura cada uno y los helpers para inferir el timeframe de un DataFrame por el delta mínimo entre timestamps. Antes de centralizar aquí estas definiciones, cada módulo tenía su propia copia, lo que causaba inconsistencias cuando se añadía un nuevo timeframe.

4.3 resampleo.py

Convierte datos de timeframe base (1h) a timeframe mayor (4h, 1d). Usa `group_by_dynamic` de Polars con ventanas cerradas a la izquierda (el timestamp visible es la apertura de la vela, igual que en TradingView). Las ventanas incompletas al principio y al final se descartan automáticamente para no contaminar con datos parciales.

La regla de agregación de cada columna es explícita y está declarada en un diccionario:

`open=primero, high=máximo, low=mínimo, close=último, volume=suma, taker_buy_volume=suma, vol_delta=suma, open_interest=último, funding_rate=último.`

Si se añade una columna nueva al histórico que no está en el diccionario, el sistema para con un error pidiendo que se declare su semántica. Esto evita que datos desconocidos se agreguen con la regla equivocada silenciosamente.

4.4 validador.py

Se ejecuta antes de cualquier operación y comprueba doce condiciones. Si alguna falla, imprime exactamente qué está mal y para el programa:

- Columnas mínimas presentes: `timestamp`, `open`, `high`, `low`, `close`.
- Columnas extra requeridas por la estrategia seleccionada.
- Sin valores nulos en precios OHLC.
- Timestamps en orden cronológico ascendente.
- Sin timestamps duplicados.
- Sin huecos temporales (configurable por activo via `MERCADO_24_7`).

- Precios coherentes: $\text{high} \geq \text{low}$, precios > 0 .
- Open y close dentro del rango $[\text{low}, \text{high}]$.

4.5 perturbaciones.py — Monte Carlo

Cuando `PERTURBACIONES_ACTIVAS = True`, cada trial de Optuna no ve los datos reales exactos sino una variante perturbada: pequeñas modificaciones aleatorias en los precios y volúmenes que producen una trayectoria de mercado alternativa pero estadísticamente plausible. El propósito es que la estrategia no aprenda a explotar ruido específico de una sola trayectoria histórica, sino que sus parámetros sean robustos ante ligeras variaciones.

Las perturbaciones están aceleradas con Numba (compilación JIT a código nativo). Cada trial recibe una semilla derivada de su número de trial, así que el resultado sigue siendo determinista si se activa la semilla maestra.

■ Las perturbaciones añaden coste de CPU por trial. Para optimizaciones exploratorias rápidas, conviene dejarlas en `False` y activarlas solo cuando se quiera endurecer un conjunto de parámetros ya prometedor.

5. MOTOR — El núcleo Rust

El motor de simulación está escrito en Rust y compilado como extensión Python mediante PyO3. Python lo importa como un módulo normal, pero por dentro es código nativo que no pasa por el intérprete Python y libera el GIL (Global Interpreter Lock), permitiendo verdadero paralelismo cuando Optuna usa múltiples workers.

wrapper.py — El puente Python ↔ Rust

Este archivo expone dos funciones principales al resto del sistema:

simular_metricas(arrays, senales, sim_cfg, salidas_custom): camino rápido. Ejecuta la simulación completa y devuelve solo el objeto Metricas con los agregados (saldo final, win rate, Sharpe, etc.). No almacena el historial de trades individuales. Es el que usa Optuna en cada trial porque es el más rápido.

simular_full(arrays, senales, sim_cfg, salidas_custom): camino completo. Devuelve Metricas + historial de trades (dict de arrays numpy) + equity_curve (array). Solo se llama en el replay de los mejores trials para generar los reportes.

La clave del rendimiento es que los arrays numpy de Python se pasan a Rust sin copia gracias al crate numpy de PyO3, que toma un puntero directo a la memoria contigua. Por eso es imprescindible que todos los arrays sean C-contiguous del dtype correcto (float64, int8, int64). La función `a_contiguo()` de `COMUN/numpy_utiles.py` garantiza esto.

Motivos de salida del motor:

Código	Nombre	Cuándo ocurre
0 - SL	Stop Loss	El precio toca el nivel de stop loss durante la vela.
1 - TP	Take Profit	El precio toca el nivel de take profit durante la vela.
2 - BARS	Tiempo (velas)	Se ha alcanzado el número máximo de velas. Cierre al precio de cierre de esa vela.
3 - CUSTOM	Señal personalizada	La estrategia emitió una señal de salida. Cierre al open de la siguiente vela (N+1).
4 - TRAILING	Trailing stop	El precio retrocede desde el máximo (o mínimo para short) más del umbral trailing.
5 - END	Fin de datos	Se acabaron los datos históricos con una posición abierta. Cierre al último close.

Regla fundamental del motor: **una señal en la barra N se ejecuta al open de la barra N+1**. Esto elimina el lookahead bias más común en backtesting: la estrategia nunca puede ver el precio de cierre de la misma vela donde genera la señal para entrar al mismo precio de cierre.

6. NUCLEO — La capa de orquestación Python

6.1 base_estrategia.py

Define la clase abstracta de la que heredan todas las estrategias. Es el contrato que deben cumplir. Incluye dos elementos de arquitectura importantes:

CacheIndicadores (LRU, 512 MB):

En una optimización de 300 trials, Optuna puede repetir el mismo valor de un parámetro de indicador (ej. EMA de periodo 21) en decenas de trials distintos. Recalcular la EMA completa en cada trial es un derroche. El caché resuelve esto: la primera vez que se pide una EMA(21) se calcula y se guarda; en el siguiente trial que pida EMA(21) se devuelve instantáneamente. El caché se limpia al cambiar de activo o timeframe para no mezclar contextos.

Método bind() / desvincular():

bind(arrays, cache) inyecta en la estrategia los arrays numpy del contexto actual (open, high, low, close, volume, timestamps) y el caché de indicadores. La estrategia no copia estos arrays: trabaja directamente sobre los punteros de memoria del contexto. desvincular() los libera al terminar para no mantener referencias colgantes.

Métodos que cada estrategia DEBE implementar:

- espacio_busqueda(trial) → define los parámetros y sus rangos para Optuna.
- generar_senales(df, params) → recibe el DataFrame del timeframe y los parámetros actuales del trial, y devuelve una pl.Series de Int8 con valores -1 (short), 0 (sin señal) o +1 (long) para cada vela.

Métodos opcionales:

- generar_salidas(df, params) → igual que generar_senales pero para señales de cierre. Solo se usa con EXIT_TYPE=CUSTOM.
- indicadores_para_grafica(df, params) → devuelve los datos de indicadores para dibujarlos en los gráficos HTML interactivos.

6.2 contexto.py

Define tres dataclasses: ArraysMotor (buffers numpy de una serie temporal), SimConfigMotor (todos los parámetros de simulación como escalares) y ContextoCombinacion (DataFrame + arrays + mapeo TF-base para una combinación activo-timeframe).

El contexto se crea una sola vez por combinación activo+timeframe y se comparte entre todos los trials. Los arrays son inmutables (frozen dataclass): ningún trial puede modificarlos.

6.3 proyeccion.py

Resuelve el problema de escala temporal: una estrategia en 4h genera una señal por cada 4 velas de 1h, pero el motor ejecuta en 1h. proyeccion.py construye un array de índices que mapea cada vela del timeframe de estrategia a la última vela del timeframe base incluida en esa ventana (el 'cierre operativo'). El motor usa este índice para saber en qué vela base ejecutar la señal.

Ejemplo: estrategia en 4h, datos base en 1h. La vela 4h que abre a las 00:00 cubre las velas 1h de 00:00, 01:00, 02:00 y 03:00. Su cierre operativo es la vela 03:00. La señal se proyecta a esa posición en el array base, y el motor entra al open de la vela 04:00.

6.4 integridad.py

Contiene todas las verificaciones cruzadas del sistema. La más importante es `verificar_resultado()`: después del replay, comprueba vectorialmente que:

- El saldo final = saldo inicial + suma de PnL de todos los trades.
- La equity curve empieza en `saldo_inicial` y termina en `saldo_final`.
- Los índices de señal, entrada y salida están en orden ($\text{señal} < \text{entrada} \leq \text{salida}$).
- La entrada siempre ocurre en la barra N+1 respecto a la señal (regla anti-lookahead).
- El precio de entrada es el open de la barra de entrada.
- Las salidas SL/TP/Trailing están dentro del rango [low, high] de su vela.
- Las salidas BARS/END usan el close; las CUSTOM usan el open de ejecución.
- Los motivos de salida son todos válidos (códigos 0-5).

Si cualquier verificación falla, hay un bug en el motor o en la proyección de señales. En 10.000 trades estas verificaciones vectoriales duran microsegundos.

6.5 registro.py

Escanea automáticamente la carpeta `ESTRATEGIAS/` en tiempo de ejecución. Cualquier clase que herede de `BaseEstrategia` y defina los atributos `ID` y `NOMBRE` queda registrada sin necesidad de declarar nada en ningún archivo de configuración. Añadir una estrategia nueva es tan sencillo como crear un archivo `.py` en esa carpeta.

7. ESTRATEGIAS — Las cinco estrategias disponibles

Cada estrategia vive en su propio archivo en ESTRATEGIAS/ y hereda de BaseEstrategia. Los indicadores se calculan dentro de cada estrategia, no en un módulo central: si dos estrategias usan una EMA pero con diferente fórmula de suavizado, ninguna afecta a la otra.

ID 1 — RSI Reversión · rsi_reversion.py · *Reversión a la media*

Usa el RSI con suavizado Wilder ($\alpha = 1/\text{periodo}$). Genera señal LONG cuando el RSI cruza al alza el nivel de sobreventa; SHORT cuando cruza a la baja el nivel de sobrecompra. La salida CUSTOM cierra cuando el RSI cruza el nivel 50. Parámetros que optimiza: periodo (7-28), nivel de sobreventa (20-40), nivel de sobrecompra (60-80). No requiere columnas extra: solo OHLC.

ID 2 — EMA Tendencia · ema_tendencia.py · *Seguimiento de tendencia*

Cruce de dos EMAs ($\alpha = 2/(\text{periodo}+1)$, sin adjust). Señal LONG cuando la EMA rápida cruza al alza la EMA lenta; SHORT en el cruce bajista. La salida CUSTOM es el cruce inverso. Parámetros: periodo_rápida (8-40), periodo_lenta (50-180). Sin columnas extra.

ID 3 — VWAP Absorption Trend · vat_absorcion.py · *Tendencial con order flow*

Combina una EWM-VWAP ponderada por volumen con una medida de absorción de order flow (vol_delta). La señal surge cuando el precio está por encima del VWAP (dirección alcista) y la presión neta de compradores supera el umbral. La lógica está compilada con Numba para máxima velocidad. Requiere columnas extra: taker_buy_volume, taker_sell_volume, vol_delta, volume. Parámetros: halflife_vwap (6-300), halflife_cvd (6-300), umbral (0.3-2.0).

ID 4 — VWAP-CVD · vwapcvd.py · *Tendencial precio + flujo de órdenes*

Dos piezas: la EWM-VWAP define la dirección operable; el CVD-Z (Cumulative Volume Delta normalizado) confirma la entrada cuando cruza su umbral. Se añade un filtro de distancia al VWAP: solo entra cuando el precio está suficientemente alejado del VWAP (zona de momentum) pero no tan lejos que sea una extensión peligrosa. Requiere taker_buy_volume, taker_sell_volume, volume.

ID 5 — VWAP Distance Reversion · vwap_distance_reversion.py · *Reversión al VWAP*

Mide la distancia normalizada del precio respecto a la EWM-VWAP. Cuando el precio está demasiado lejos del VWAP en términos de desviaciones estándar (distance_z), espera el retorno al umbral como señal de reversión. La señal se activa por cruce del umbral (no por nivel): primero el indicador debe superar el umbral y luego volver a cruzarlo, evitando entradas prematuras. Requiere volume.

Para añadir una estrategia nueva: crear ESTRATEGIAS/mi_estrategia.py, definir ID, NOMBRE y COLUMNAS_REQUERIDAS únicos, heredar de BaseEstrategia, e implementar espacio_busqueda() y generar_senales(). El sistema la detectará automáticamente en el siguiente run.

8. SALIDAS — Los cuatro tipos de cierre

El tipo de salida activo se configura con `EXIT_TYPE` en `config.py`. Cada tipo tiene su propio archivo de parámetros en `SALIDAS/` y puede activar la optimización de sus propios parámetros con `OPTIMIZAR_SALIDAS = True`.

FIXED — Stop Loss y Take Profit fijos · `fijo.py`

- `EXIT_SL_PCT`: porcentaje de pérdida sobre el colateral que cierra el trade (ej. 15 → cierra al perder el 15% del colateral).
- `EXIT_TP_PCT`: porcentaje de ganancia sobre el colateral que cierra el trade (ej. 25 → cierra al ganar el 25%).
- El motor evalúa en cada vela si el precio ha tocado el nivel SL o TP dentro del rango `[low, high]`. Si ambos se tocan en la misma vela, gana el que se toca primero según la dirección del movimiento.
- Cuando `OPTIMIZAR_SALIDAS=True`, Optuna también busca los valores óptimos de SL y TP dentro de los rangos `EXIT_SL_MIN/MAX` y `EXIT_TP_MIN/MAX`.

BARS — Cierre por tiempo (número de velas) · `velas.py`

- `EXIT_VELAS`: número de velas que puede durar el trade. Al alcanzarlo, se cierra al precio de close.
- `USAR_SL_EMERGENCIA`: permite activar un SL de seguridad en paralelo. Si el precio lo toca antes de completar las velas, cierra por SL.
- Útil para estrategias que no definen niveles de precio explícitos pero sí tienen una idea de cuánto tiempo debería durar una operación.
- Cuando `OPTIMIZAR_SALIDAS=True`, Optuna busca el número óptimo de velas.

TRAILING — Trailing stop · `trailing.py`

- `EXIT_SL_PCT`: stop de seguridad que protege en todo momento. Si el precio lo toca antes de activarse el trailing, cierra por SL.
- `EXIT_TRAIL_ACT_PCT`: beneficio mínimo sobre el colateral para activar el trailing (ej. 30 → se activa cuando el trade gana un 30%).
- `EXIT_TRAIL_DIST_PCT`: distancia del trailing respecto al mejor precio alcanzado (ej. 6 → si el precio retrocede un 6% desde el máximo del trade, se cierra).
- El trailing protege ganancias sin fijar un objetivo: deja correr las posiciones ganadoras hasta que el mercado decide revertir.

CUSTOM — Salida definida por la estrategia · `personalizada.py`

- La condición de cierre la define la propia estrategia en su método `generar_salidas()`. Puede ser cualquier lógica de indicadores.
- `EXIT_SL_PCT`: SL de seguridad activo en paralelo. Si el precio lo toca antes de la señal de salida, el motor cierra por SL.
- La salida CUSTOM se confirma en la vela N y se ejecuta al open de la vela N+1 (misma regla anti-lookahead que las entradas).
- Ejemplos: RSI Reversión usa como salida el cruce del RSI por 50; EMA Tendencia usa el cruce inverso de EMAs.

9. OPTIMIZACION — El cerebro del sistema

9.1 runner.py — Orquestador principal

runner.py contiene la función `main()` y es el punto de entrada del sistema. Orquesta todo el flujo: carga configuración, registra estrategias, itera sobre activos × timeframes × salidas, lanza la optimización de cada combinación y genera todos los reportes.

Dataclasses clave:

- **ExitConfig:** Encapsula la configuración de salida de un run: tipo, `sl_pct`, `tp_pct`, velas, parámetros de trailing y si se optimizan o no los parámetros de salida.
- **TrialResultado:** Resultado completo de un trial: número, activo, timeframe, ID de estrategia, configuración de salida, parámetros del trial, score, métricas, conteo de señales y datos de replay.
- **ReplayTrial:** Datos del replay de un trial: objeto `Metricas` completo, dict de arrays de trades (numpy), y `equity_curve` (numpy).

Flujo interno de `_optimizar_combinacion()`:

1. Crea un estudio Optuna con el sampler elegido (ver `samplers.py`).
2. Define la función objetivo (closure) que para cada trial: hace `bind()` de arrays y caché a la estrategia, genera señales, proyecta al timeframe base, llama a `simular_metricas()`, calcula el score con `puntuacion.py`.
3. Si `PERTURBACIONES_ACTIVAS=True`, clona la estrategia por trial y aplica las perturbaciones antes de generar señales.
4. Lanza `study.optimize()` con `n_jobs` paralelos. El motor Rust libera el GIL, por eso el paralelismo es real.
5. Al finalizar, identifica los top-N trials y llama a `_replay_trial()` sobre cada uno con `simular_full()`.
6. Llama a `verificar_resultado()` de `integridad.py` para confirmar determinismo.
7. Calcula `analitica_avanzada()` y `tabla_monthly_returns()` sobre los datos del replay.
8. Guarda todo con `persistencia.py` y genera reportes PDF, Excel, HTML e informe.

9.2 metricas.py — Puente Rust → Python

El motor Rust devuelve un objeto `Metricas` con los agregados de la simulación. `metricas.py` lo convierte en un diccionario Python y añade las métricas derivadas que no están en el struct Rust: `trades_por_dia`, CAGR, Calmar, Sharpe anualizado y PSR (aproximación rápida usando solo la distribución normal, sin skew/kurtosis, para no añadir coste a los 300 trials).

9.3 puntuacion.py — Función objetivo

Optuna maximiza el valor que devuelve esta función. Hay cuatro opciones:

- **PSR:** Probabilistic Sharpe Ratio [0, 1]. Penaliza estrategias con pocos trades o alta curtosis. Es la función más robusta porque un Sharpe alto con muestra pequeña puede ser pura suerte.
- **SHARPE:** Sharpe ratio anualizado. Simple y directo, pero no penaliza la muestra pequeña.
- **CALMAR:** CAGR / Max Drawdown. Prioriza retornos altos con drawdowns controlados.
- **ROI:** Retorno total sobre el saldo inicial. Maximiza rentabilidad pura sin ajustar por riesgo.

En todos los casos, si el trial tiene menos de `MIN_TRADES_SCORE` operaciones, el score es 0. Esto evita que Optuna converja hacia configuraciones que hacen muy pocas operaciones (cuyo Sharpe puede ser artificialmente alto por puro azar).

9.4 samplers.py — Algoritmos de búsqueda

- **QMC (Quasi-Monte Carlo):** Muestrea el espacio de parámetros con secuencias cuasi-aleatorias que cubren el espacio de manera más uniforme que el muestreo aleatorio puro. Ideal para exploración inicial: en 300 trials visita más del espacio que el TPE bayesiano que necesita exploración previa.
- **TPE (Tree-structured Parzen Estimator):** Modelo bayesiano que construye un árbol de densidad de probabilidad basado en los trials anteriores y propone los siguientes trials en las zonas del espacio que han dado mejores resultados. Excelente para explotación (refinar) pero necesita unos 50-100 trials de exploración previa.
- **HYBRID:** Primera mitad de trials con QMC (exploración), segunda mitad con TPE (explotación). Es el mejor de los dos mundos para la mayoría de optimizaciones.

10. COMUN — Utilidades compartidas

estadistica.py

Fuente única de verdad para todas las fórmulas estadísticas del sistema. Cualquier módulo que necesite calcular un Sharpe, un PSR o un CAGR importa de aquí. Esto evita que la misma fórmula exista en varios sitios con ligeras diferencias.

- `phi(x)`: CDF de la distribución normal estándar. Base del PSR.
- `sharpe_anualizado(sr, tpy)`: $SR \times \sqrt{\text{trades_por_año}}$. Escala el Sharpe por trade a anual.
- `probabilistic_sharpe_ratio(sr, n, ref, skew, kurt)`: $PSR = \Phi[(SR - SR_ref) \times \sqrt{(n-1)} / \sqrt{(1 - skew \times SR + (kurt-1)/4 \times SR^2)}]$. Corrección completa con asimetría y curtosis (Bailey & López de Prado, 2012). Devuelve la probabilidad de que el Sharpe verdadero supere `SR_ref`.
- `cagr(si, sf, años)`: $((sf/si)^{(1/años)}) - 1$. Tasa de crecimiento anual compuesta.
- `calmar(cagr, max_dd)`: CAGR / MaxDrawdown. Ratio rentabilidad-drawdown.
- `sortino_por_trade(media, desv_bajista)`: `media / desv_bajista`. Penaliza solo la volatilidad negativa.

numpy_utiles.py

Contiene una única función: `a_contiguo(arr, dtype)`. Devuelve el array como ndarray C-contiguo del dtype pedido. Si el array ya cumple ambas condiciones, lo devuelve sin copiar. Si no, hace una única copia. Esto es imprescindible porque el motor Rust necesita arrays contiguos en memoria para acceder a ellos sin copia adicional vía PyO3.

11. REPORTES — Todo lo que se genera

Al terminar una optimización, el sistema guarda todos los resultados en una carpeta de run con estructura: RUNS/ACTIVO_TIMEFRAME ESTRATEGIA_SALIDA_YYYYMMDD_HHMMSS/

11.1 analitica.py — Fuente única de métricas avanzadas

La función `analitica_avanzada(metricas, trades, equity_curve, fecha_inicio, fecha_fin)` es la fuente única de verdad para todas las métricas avanzadas. Tanto el PDF como el Excel consumen desde aquí; no hay cálculos duplicados.

Categorías de métricas calculadas aquí:

- **Retorno ajustado por riesgo:** Sortino anualizado, PSR completo (con skew/kurtosis), Calmar, CAGR, Recovery factor.
- **Distribución de PnL:** VaR 95%, CVaR 95%, Asimetría, Curtosis, Percentiles 5/25/50/75/95.
- **Rendimiento:** Profit bruto y neto, `gross_profit`, `gross_loss`, `avg_win`, `avg_loss`, mejor/peor trade.
- **Exposición y actividad:** % tiempo en mercado, trades por día, medias diarias/mensuales/anuales de PnL.
- **Rachas:** Máxima racha ganadora/perdedora, media de rachas ganadoras/perdedoras.
- **Estadística de trades:** Z-Score (Wald-Wolfowitz), SQN (Van Tharp), R-Expectancy, AHPR, GHPR.
- **Stagnation:** Días y porcentaje del tiempo total que el equity estuvo en drawdown (sin hacer nuevos máximos).
- **Retornos mensuales:** `tabla_monthly_returns()`: PnL agrupado por año y mes (por fecha de salida del trade).

11.2 reporte_pdf.py — El informe en PDF

Genera un PDF de 2 páginas en formato A4 horizontal con reportlab (Platypus + Canvas) y matplotlib.

Página 1:

Cabecera con nombre del trial, activo, timeframe y estrategia. Aviso en ámbar de que los resultados son in-sample. Fila de parámetros óptimos del trial. Grid de 3×6 KPIs con sistema semáforo (verde/ámbar/rojo según umbrales). Tabla de dos columnas con todas las estadísticas avanzadas.

Página 2:

Gráfico de curva de equity + drawdown generado con matplotlib y embebido como imagen PNG. Tabla de retornos mensuales con colores proporcionales a la magnitud (verde positivo, rojo negativo).

11.3 excel.py — Informe Excel

Genera un archivo Excel con una hoja por trial (top-5 por defecto). Cada hoja contiene la tabla completa de trades, la curva de equity como gráfico y el panel de métricas con valoraciones BUENO/REGULAR/MALO. Usa `xlsxwriter` para el formato.

11.4 html.py — Gráficos interactivos TradingView

Genera hasta `MAX_PLOTS` archivos HTML con gráficos interactivos usando la biblioteca TradingView Lightweight Charts. Cada HTML muestra el gráfico de precios con los indicadores de la estrategia superpuestos, las marcas de entrada y salida de cada trade, y la curva de equity. La biblioteca se embebe localmente (se descarga una vez y se guarda en `assets/`).

11.5 informe.py — Paisaje de parámetros

Genera un informe HTML con Plotly que muestra todos los trials de la optimización: distribución de scores, gráficos de dispersión de parámetros vs score, y coordenadas paralelas para identificar qué combinaciones de parámetros dan mejores resultados. Permite detectar si la optimización convergió o si hay muchas combinaciones igualmente buenas.

11.6 persistencia.py — Archivos permanentes

- `resumen.json`: Resumen completo de la optimización: parámetros, métricas del mejor trial, fechas.
- `auditoria.json`: Registro de verificaciones de integridad y determinismo.
- `trials.csv`: Tabla con todos los trials: score, parámetros, métricas.
- `trades.csv`: Historial completo de trades del mejor trial con todos los campos.
- `equity.csv`: Curva de equity punto a punto.
- `PDF/REPORTE TRIAL N.pdf`: El informe PDF para los top-N trials.

11.7 rich.py — Terminal en tiempo real

Muestra el progreso de la optimización en la terminal con colores y tablas usando la biblioteca Rich. Durante la optimización muestra el progreso de Optuna. Al finalizar, muestra un resumen con las métricas principales del mejor trial y la ruta al PDF generado.

11.8 metricas_presentacion.py — Etiquetas y juicios

Define los umbrales de calificación para cada métrica (qué nivel es BUENO, REGULAR o MALO) y calcula un veredicto global ponderado. El veredicto tiene dos ejes:

- **Fiabilidad (PSR)**: ¿el edge es real o puede ser ruido estadístico?
- **Calidad**: combinación ponderada de Sharpe anualizado (40%), Calmar (25%), control de drawdown (20%) y Profit Factor (15%).

Nota final = Fiabilidad × Calidad. Un sistema con alta rentabilidad pero PSR bajo recibirá nota baja porque el resultado podría ser sobreajuste.

12. Guía completa de métricas

Todas las métricas del sistema, su fórmula, interpretación y umbrales de calificación.

Métricas de rentabilidad

Métrica	Fórmula / cálculo	Interpretación	Umbral orientativo
ROI Total	$\frac{\text{PnL_total}}{\text{Saldo_inicial}}$	Retorno total sobre el capital inicial. 0.5 = 50% de beneficio.	—
CAGR	$\left(\frac{\text{SF}}{\text{SI}}\right)^{\frac{1}{\text{años}}} - 1$	Tasa de crecimiento anual compuesta. Permite comparar estrategias con horizontes distintos.	≥15% = bueno
Profit Factor	$\frac{\text{Suma_ganancias}}{\text{Suma_pérdidas}}$	Cuánto ganas por cada unidad que pierdes. >1.5 es bueno; <1.0 es estrategia perdedora.	>1.5 = bueno
Expectancy	$\frac{\text{Win_rate} \times \text{avg_win} - (1 - \text{Win_rate}) \times \text{avg_loss}}{1}$	Ganancia esperada por operación en promedio. Debe ser positiva.	>0 = mínimo
Payoff Ratio	$\frac{\text{avg_win}}{\text{avg_loss}}$	Tamaño medio de ganadores vs perdedores. >1 significa que cada ganador supera al perdedor medio.	>2.0 = bueno
Gross Profit / Loss	Suma de PnL positivos / negativos	Beneficio y pérdida brutos antes de netear. Permite ver escala del trading.	—
Yearly / Monthly Avg PnL	$\frac{\text{PnL_total}}{\text{años y meses}}$	Cuánto genera el sistema por año/mes en promedio.	—

Métricas de riesgo y drawdown

Métrica	Fórmula / cálculo	Interpretación	Umbral orientativo
Max Drawdown (%)	$\frac{\text{Max}(\text{peak} - \text{equity})}{\text{peak}} \times 100$	Caída máxima desde un pico de equity hasta el siguiente mínimo. El riesgo más intuitivo.	<10% = bueno
Max Drawdown (\$)	$\text{Max}(\text{peak} - \text{equity})$ en unidades monetarias	Mismo drawdown en valor absoluto. Más útil para calibrar el capital necesario.	—
Calmar Ratio	$\frac{\text{CAGR}}{\text{Max_Drawdown}}$	Rentabilidad anual por unidad de drawdown máximo. >3 = excelente.	>3.0 = bueno
Recovery Factor	$\frac{\text{ROI_total}}{\text{Max_Drawdown}}$	Cuántas veces el retorno total supera el mayor drawdown. >3 = robusto.	>3 = bueno
Return/DD Ratio	$\frac{\text{ROI_total}}{\text{Max_Drawdown}}$	Similar a recovery factor pero sobre retorno total.	—
Stagnation (días)	Días totales en drawdown	Tiempo total que el equity pasó sin marcar nuevos máximos. Captura el desgaste psicológico.	—
Stagnation (%)	$\frac{\text{Stagnation_días}}{\text{Total_días}}$	Porcentaje del horizonte analizado en el que el sistema estaba recuperándose.	<50% = deseable

Métrica	Fórmula / cálculo	Interpretación	Umbral orientativo
VaR 95%	Percentil 5 del PnL por trade	La pérdida que no se supera en el 95% de los trades. Interpretación: en el 5% peor de los trades, la pérdida es mayor que este valor.	—
CVaR 95%	Media del PnL en el peor 5%	También llamado Expected Shortfall. Promedio de pérdidas en el 5% peor de los trades. Más conservador que el VaR.	—

Métricas ajustadas por riesgo

Métrica	Fórmula / cálculo	Interpretación	Umbral orientativo
Sharpe Ratio (anualizado)	$SR_{por_trade} \times \sqrt{(trades_por_año)}$	Rentabilidad por unidad de volatilidad total, escalado a anual. Estándar del sector. >1 = aceptable, >2 = excelente.	>2.0 = excelente
Sortino Ratio (anualizado)	$\frac{media_PnL}{desv_bajista} \times \sqrt{(tpy)}$	Como el Sharpe pero penalizando solo la volatilidad de los trades perdedores, no la de los ganadores. Más justo para estrategias con alta dispersión positiva.	>2.5 = bueno
PSR Probabilistic Sharpe Ratio	$\Phi[(SR - SR_{ref}) \times \sqrt{(n-1)} / corrección_skew_kurt]$	Probabilidad de que el Sharpe verdadero supere el umbral de referencia (0 por defecto). Incorpora corrección por tamaño de muestra, asimetría y curtosis. >90% = alta confianza.	>0.9 = verde

Métricas estadísticas de trades

Métrica	Fórmula / cálculo	Interpretación	Umbral orientativo
Z-Score (Wald-Wolfowitz)	$z = (R - R_{esperado}) / \sigma_R$	Testa si la secuencia de trades ganadores/perdedores es aleatoria o tiene dependencia. $ Z > 1.96$ al 95% indica dependencia (rachas).	$ Z < 1.96$ = independencia
SQN (Van Tharp)	$\sqrt{n} \times media_PnL / desv_PnL$	System Quality Number. Mide la consistencia del edge. <1.6 = pobre, 1.6-2.5 = aceptable, >2.5 = bueno, >5 = excelente.	>2.5 = bueno
R-Expectancy	$(WR \times payoff - (1-WR)) / payoff$	Expectativa en unidades de R (riesgo). 0.2 = por cada unidad de riesgo se espera ganar 0.2 en media.	>0 = necesario
R-Expectancy Score	$R-Exp \times \sqrt{(n/n_{ref})}$	R-Expectancy ponderada por la muestra. Penaliza cuando hay pocos trades.	—
AHPR	$1 + media(PnL/saldo)$	Arithmetic Holding Period Return. Retorno aritmético medio por trade.	>1 = positivo
GHPR	$(\prod(1 + PnL_i/saldo))^{(1/n)}$	Geometric Holding Period Return (Vince 1992). Retorno geométrico compuesto por trade. Más realista que el aritmético al reinvertir.	>1 = positivo

Métrica	Fórmula / cálculo	Interpretación	Umbral orientativo
Asimetría (Skew)	tercer momento normalizado	>0 = más trades pequeños perdedores y pocos ganadores grandes (bueno para mantener). <0 = lo contrario.	>0 = deseable
Curtosis (Kurtosis)	cuarto momento normalizado	Exceso de colas respecto a la normal. >3 = colas gordas (el peor trade es muy peor de lo esperado).	~1-3 = normal

Métricas de actividad

Métrica	Fórmula / cálculo	Interpretación	Umbral orientativo
Win Rate	$\text{Trades_ganadores} / \text{Total_trades}$	Porcentaje de operaciones con PnL positivo. No tiene que ser >50% si el payoff ratio es alto.	>50% = deseable
Total Trades	Conteo de operaciones cerradas	Número de operaciones. Más trades = estimaciones más fiables estadísticamente.	>30 = mínimo
Trades Long / Short	Desglose por dirección	Permite ver si la estrategia tiene sesgo direccional.	—
Exposición (%)	$\text{Velas_con_posición} / \text{Total_velas}$	Porcentaje del tiempo que el sistema tiene posición abierta.	—
Duración media (velas)	Media de velas que dura cada trade	Cuánto tiempo permanece abierta una operación en media.	—
Max Racha Ganadora / Perdedora	Secuencia más larga de resultados iguales	La racha perdedora máxima es el test de resistencia psicológica del sistema.	—

13. Limitación In-Sample y próximos pasos

■ ADVERTENCIA CRÍTICA: TODOS LOS RESULTADOS SON IN-SAMPLE

Los parámetros óptimos que muestra el sistema se encontraron buscando en los mismos datos históricos sobre los que se miden los resultados. Esto significa que **PARTE** del rendimiento que se reporta puede no ser habilidad real de la estrategia, sino adaptación a patrones específicos de ese periodo concreto de datos (sobreajuste u overfitting).

Para obtener una estimación honesta del rendimiento fuera de muestra es necesario implementar alguna de las siguientes técnicas. Ninguna elimina el riesgo de sobreajuste completamente, pero reduce enormemente la probabilidad de publicar un resultado inflado.

1. Walk-Forward Analysis (WFA) — Pendiente de implementar

La técnica más robusta y la recomendada como siguiente paso. El horizonte histórico se divide en ventanas. Para cada ventana:

- Se optimiza la estrategia en la ventana In-Sample (IS).
- Los parámetros óptimos IS se aplican sin reoptimizar en la ventana Out-of-Sample (OOS) inmediatamente posterior.
- La métrica OOS es la medición honesta del rendimiento.
- Se avanza la ventana y se repite el proceso (rolling o anchored).
- Los resultados OOS de todas las ventanas se concatenan para formar la curva de equity OOS.

Si la curva OOS se parece a la IS, el edge es robusto. Si el rendimiento colapsa fuera de muestra, la estrategia estaba sobreajustada al periodo IS.

2. Perturbaciones Monte Carlo (ya disponible)

Activar `PERTURBACIONES_ACTIVAS = True` en `config.py`. Cada trial verá datos ligeramente distintos, forzando a Optuna a encontrar parámetros que funcionen en múltiples trayectorias posibles, no solo en la trayectoria exacta observada. Reduce sobreajuste a trayectoria pero no sustituye al WFA para validación OOS.

3. Análisis de Robustez de Parámetros

El informe HTML de parámetros (`informe.py`) permite ver si los parámetros óptimos están en una meseta plana (varios valores cercanos dan resultados similares = robusto) o en un pico muy estrecho (solo esos valores exactos funcionan = sospechoso de sobreajuste).

4. PSR como filtro de fiabilidad

El PSR incorpora corrección por tamaño de muestra: un Sharpe obtenido con 30 trades tiene una incertidumbre enorme. $PSR > 0.9$ significa que la probabilidad estadística de que el edge sea mayor que cero supera el 90%. Es una defensa parcial contra el sobreajuste por muestra pequeña, pero no contra el sobreajuste a la trayectoria histórica.

Recomendación de uso del sistema actual:

Los resultados IS son útiles para descartar estrategias que claramente no funcionan ni con la mejor configuración posible, para explorar el espacio de parámetros y entender la sensibilidad de la estrategia, y para comparar estrategias entre sí sobre el mismo activo y periodo. No son útiles para predecir el rendimiento futuro real sin validación OOS adicional.

Documento generado automáticamente el 27 de June de 2026. Versión del sistema: PANEL BACKTESTING con motor Rust (PyO3) + Optuna + Polars + Reportlab.